

Week 9

Graph Algorithms

Order · Weights · Cycles · Backtracking

Today's map

- Order: topological sort (2 ways), cycle detection
- Track groups: union-find
- Cheapest connections: Kruskal, Prim (MST)
- Cheapest paths: Dijkstra, Bellman-Ford, Floyd-Warshall
- Reachability: strongly connected components, bipartite
- Beyond paths: max flow, backtracking

Before we start: what you know

- Week 5: adjacency list, alphabetical neighbours
- BFS: queue, fewest edges
- DFS: recursion or explicit stack
- Connected components: BFS/DFS from every unvisited vertex

The one new ingredient: weights

- 6 of today's algorithms attach a **number** to every edge
- A distance, a cost, a capacity
- The question changes from "reachable?" to "**cheapest?**"

What this week reuses from Week 5

Week 5 tool	Reused by
Adjacency list, alphabetical order	Every algorithm this week
BFS queue	Bipartite check, max-flow's augmenting path
DFS recursion	Topo-sort, cycle detection, SCC, backtracking
Circle layout	Every animation's graph drawing

1. Topological Sort

A question to start

Socks before shoes. Shirt before jacket.
No order between socks and shirt.

Is there always one order obeying every rule?

A short history

- **A. B. Kahn**, 1962 — in-degree + queue
- Build systems, package installers use this daily
- The DFS-based alternative falls out of **Tarjan's** 1970s DFS framework

Kahn's idea

- A vertex with **in-degree 0** has no unmet prerequisite
- Place it now; this removes its outgoing edges
- Every neighbour's in-degree drops by one
- Newly-zero neighbours become eligible — enqueue them

Kahn's code (C) — part 1/2

```
int topo_sort_kahn(Graph *g) {
    for (int i=0; i<g->vertex_count; i++) indeg[i]=0;
    for (int u=0; u<g->vertex_count; u++)
        for (AdjNode *n=g->adj[u]; n; n=n->next)
            indeg[n->to]++;
    front=rear=0; order_len=0;
    for (int v=0; v<g->vertex_count; v++)
        if (indeg[v]==0) enqueue(v);
}
```

Kahn's code (C) — part 2/2

```
while (front<rear) {  
    int u=dequeue();  
    order[order_len++]=u;  
    for (AdjNode *n=g->adj[u]; n; n=n->next) {  
        indeg[n->to]--;  
        if (indeg[n->to]==0) enqueue(n->to);  
    }  
}  
return order_len==g->vertex_count;  
}
```

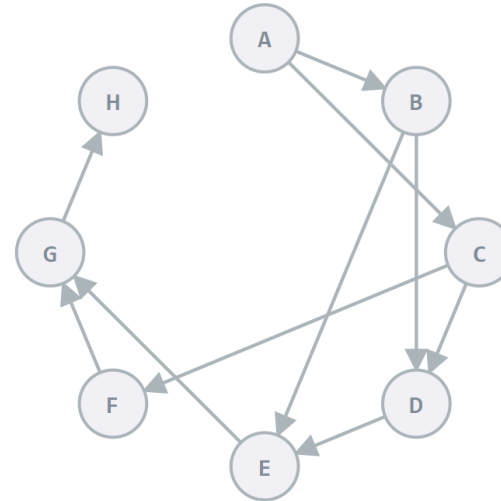
Kahn's code (Java) — part 1/2

```
static boolean topoSortKahn(Graph g) {  
    for (int i=0; i<g.vertexCount; i++) indeg[i]=0;  
    for (int u=0; u<g.vertexCount; u++)  
        for (AdjNode n=g.adj[u]; n!=null; n=n.next)  
            indeg[n.to]++;  
    front=rear=0; orderLen=0;  
    for (int v=0; v<g.vertexCount; v++)  
        if (indeg[v]==0) enqueue(v);  
}
```

Kahn's code (Java) — part 2/2

```
while (front<rear) {  
    int u=dequeue();  
    order[orderLen]=u; orderLen++;  
    for (AdjNode n=g.adj[u]; n!=null; n=n.next) {  
        indeg[n.to]--;  
        if (indeg[n.to]==0) enqueue(n.to);  
    }  
}  
return orderLen==g.vertexCount;  
}
```

Kahn's algorithm



indeg[] =

A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	0

queue =

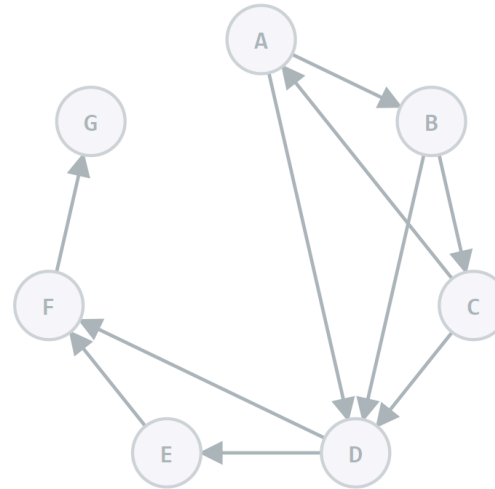
--	--	--	--	--	--	--	--

order =

A	B	C	D	F	E	G	H
---	---	---	---	---	---	---	---

The queue is empty, all 8 vertices were placed: A, B, C, D, F, E, G, H. Every vertex comes after all of its prerequisites.

Edge case: a cycle



indeg[] =

A	B	C	D	E	F	G
1	1	1	3	1	2	1

queue =

--	--	--	--	--	--	--

order =

The queue is empty but A, B, C, D, E, F, G never got placed -- they form a CYCLE whose in-degree never reaches 0. No topological order exists.

Real output: Kahn's algorithm

```
-- normal: 8 vertices, 10 edges, a valid DAG --  
order: A B C D F E G H  
all 8 vertices placed: a valid topological order  
  
-- edge: 10 edges but a cycle exists, no full order --  
order:  
only 0 of 7 vertices placed -- a cycle exists
```

The DFS idea

- Run recursive DFS (Week 5's DFS, one array added)
- Record every vertex's **finish time**
- Read finish times **largest to smallest**
- A vertex finishes only after everything it points to

DFS topo-sort code (C) — dfs_visit

```
void dfs_visit(int u) {  
    color_of[u]=1;  
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next) {  
        if (color_of[n->to]==0) dfs_visit(n->to);  
        else if (color_of[n->to]==1) has_cycle=1;  
    }  
    color_of[u]=2;  
    finish[finish_len++]=u;  
}
```

DFS topo-sort code (C) — the driver

```
void topo_sort_dfs(Graph *g) {  
    cur_g=g;  
    for (int i=0;i<g->vertex_count;i++) color_of[i]=0;  
    finish_len=0; has_cycle=0;  
    for (int v=0;v<g->vertex_count;v++)  
        if (color_of[v]==0) dfs_visit(v);  
}
```

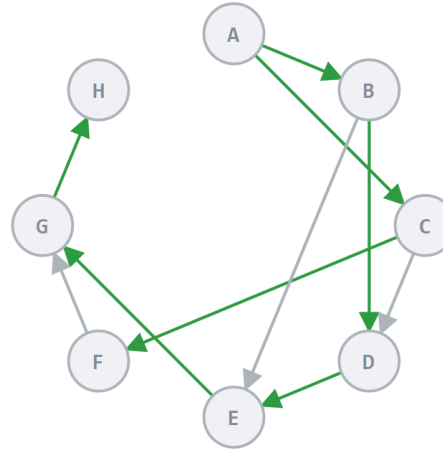
DFS topo-sort code (Java) — dfsVisit

```
static void dfsVisit(int u) {  
    colorOf[u]=1;  
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next) {  
        if (colorOf[n.to]==0) dfsVisit(n.to);  
        else if (colorOf[n.to]==1) hasCycle=true;  
    }  
    colorOf[u]=2;  
    finish[finishLen]=u; finishLen++;  
}
```

DFS topo-sort code (Java) — the driver

```
static void topoSortDfs(Graph g) {  
    curG=g;  
    for (int i=0;i<g.vertexCount;i++) colorOf[i]=0;  
    finishLen=0; hasCycle=false;  
    for (int v=0;v<g.vertexCount;v++)  
        if (colorOf[v]==0) dfsVisit(v);  
}
```

DFS topological sort



color[] =

A	B	C	D	E	F	G	H
2	2	2	2	2	2	2	2

call stack =

--	--	--	--	--	--	--	--

finish order =

H	G	E	D	B	F	C	A
---	---	---	---	---	---	---	---

topo order =

A	C	F	B	D	E	G	H
---	---	---	---	---	---	---	---

The finish order is reversed: A, C, F, B, D, E, G, H. Every vertex comes after all of its incoming edges.

Complexity & mistake

- $O(V + E)$ — both algorithms, one pass
- Mistake: a topological order is **not unique**
- Compare **positions**, never hard-code the exact sequence

Mini question

Why does Kahn's algorithm use a **queue**, not just one recursive call?

(answer on the next slide)

Answer

A DAG can have **several independent sources** at once.

The queue lets the algorithm interleave them in one pass, instead of finishing one branch before starting another.

2. Cycle Detection (Directed)

A question to start

Section 1's DFS detects **that** a cycle exists (a back edge).

Which vertices form it, in what order?

3-colour DFS with a path

- White / **gray** (on the path) / black (finished)
- Keep the current path in `on_path[]`
- A back edge to a **gray** vertex: that vertex is an open ancestor
- The path slice from there to here **IS** the cycle

Cycle detection code (C) — part 1/2

```
int dfs_cycle(int u) {  
    color_of[u]=1;  
    on_path[path_top++]=u;  
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next) {  
        if (color_of[n->to]==0) {  
            if (dfs_cycle(n->to)) return 1;  
        } else if (color_of[n->to]==1) {
```

Cycle detection code (C) — part 2/2

```
        int i=path_top-1;
        while (on_path[i]!=n->to) i--;
        cycle_len=0;
        for (; i<path_top; i++)
            cycle[cycle_len++]=on_path[i];
        return 1;
    }
}
path_top--; color_of[u]=2;
return 0;
}
```

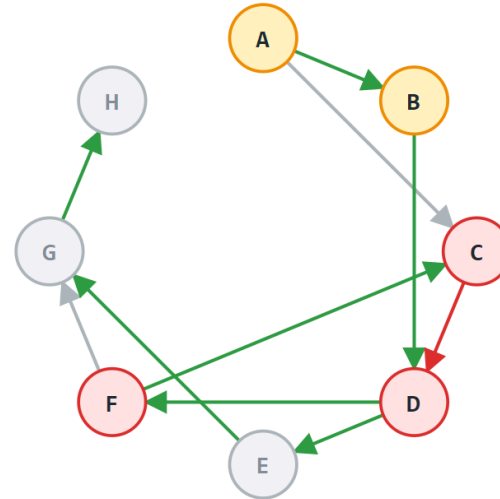
Cycle detection code (Java) — part 1/2

```
static boolean dfsCycle(int u) {  
    colorOf[u]=1;  
    onPath[pathTop]=u; pathTop++;  
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next) {  
        if (colorOf[n.to]==0) {  
            if (dfsCycle(n.to)) return true;  
        } else if (colorOf[n.to]==1) {
```

Cycle detection code (Java) — part 2/2

```
        int i=pathTop-1;
        while (onPath[i]!=n.to) i--;
        cycleLen=0;
        for (; i<pathTop; i++) {
            cycle[cycleLen]=onPath[i]; cycleLen++;
        }
        return true;
    }
    pathTop--; colorOf[u]=2;
    return false;
}
```

Cycle detection, directed graph



color[] =

A	B	C	D	E	F	G	H
1	1	1	1	2	1	2	2

path (on_path) =

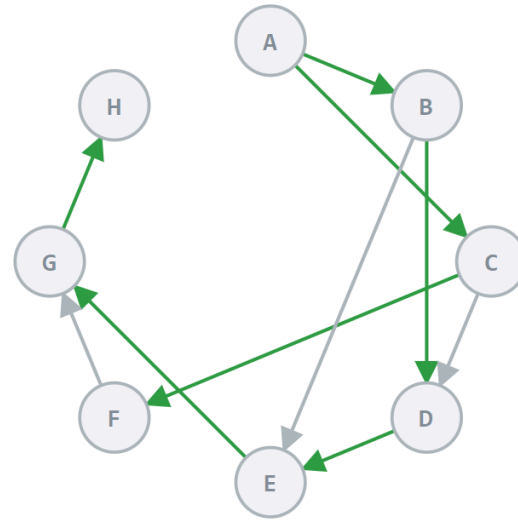
A	B	D	F	C			
---	---	---	---	---	--	--	--

cycle =

D	F	C
---	---	---

Neighbour D is grey: D is still on the path, an ANCESTOR. The slice of the path from there to here IS A CYCLE:
D -> F -> C -> D.

Edge case: no cycle at all



color[] =

A	B	C	D	E	F	G	H
2	2	2	2	2	2	2	2

path (on_path) =

--	--	--	--	--	--	--	--

cycle =

Every vertex is black, no back edge was ever found: this graph HAS NO CYCLE.

Real output: cycle detection

```
-- normal: 8 vertices, 10 edges, one cycle: C-D-F-C --  
cycle found: D F C -> D  
  
-- edge: 10 edges, entirely cycle-free (a DAG) --  
no cycle found  
  
-- edge: the smallest cycle, A-B-A (2 edges) --  
cycle found: A B -> A
```

Complexity & mistake

- $O(V + E)$ — one DFS, plus $O(V)$ to extract a found cycle
- Mistake: checking "not white" instead of "**gray**"
- A black neighbour is finished and safe — not a cycle

Mini question

Section 1's DFS topo-sort just sets `has_cycle = 1`. Why does THIS algorithm need the extra `on_path[]` stack at all?

(answer on the next slide)

Answer

Knowing a cycle exists is not the same as knowing **which vertices** form it.

`on_path[]` keeps the current root-to-here chain available, so the moment a back edge is found, the cycle can be read straight off it.

3. Union-Find

A question to start

Kruskal's algorithm (coming up) must ask, over and over:

"Are these two vertices already connected by edges I picked?"

A fresh BFS every time would be correct — but slow.

A forest of parent pointers

- `find(v)` walks parent pointers up to the **root**
- Same root = same group
- `union(a, b)` merges two groups: root points to root

Two tricks

- **Union by rank:** shorter tree hangs under the taller
- **Path compression:** `find` re-points every visited node straight at the root
- Together: $O(\alpha(n))$ per operation — for all practical `n`, this is $O(1)$

Union-find code (C) — find

```
int find(int v) {  
    int root=v;  
    while (parent_of[root]!=root) root=parent_of[root];  
    while (parent_of[v]!=root) {  
        int next=parent_of[v];  
        parent_of[v]=root;  
        v=next;  
    }  
    return root;  
}
```

Union-find code (C) — union_sets

```
void union_sets(int a, int b) {  
    int ra=find(a), rb=find(b);  
    if (ra==rb) return;  
    if (rank_of[ra]<rank_of[rb]) parent_of[ra]=rb;  
    else if (rank_of[ra]>rank_of[rb]) parent_of[rb]=ra;  
    else { parent_of[rb]=ra; rank_of[ra]++; }  
}
```

Union-find code (Java) — find

```
static int find(int v) {  
    int root=v;  
    while (parentOf[root]!=root) root=parentOf[root];  
    while (parentOf[v]!=root) {  
        int next=parentOf[v];  
        parentOf[v]=root;  
        v=next;  
    }  
    return root;  
}
```

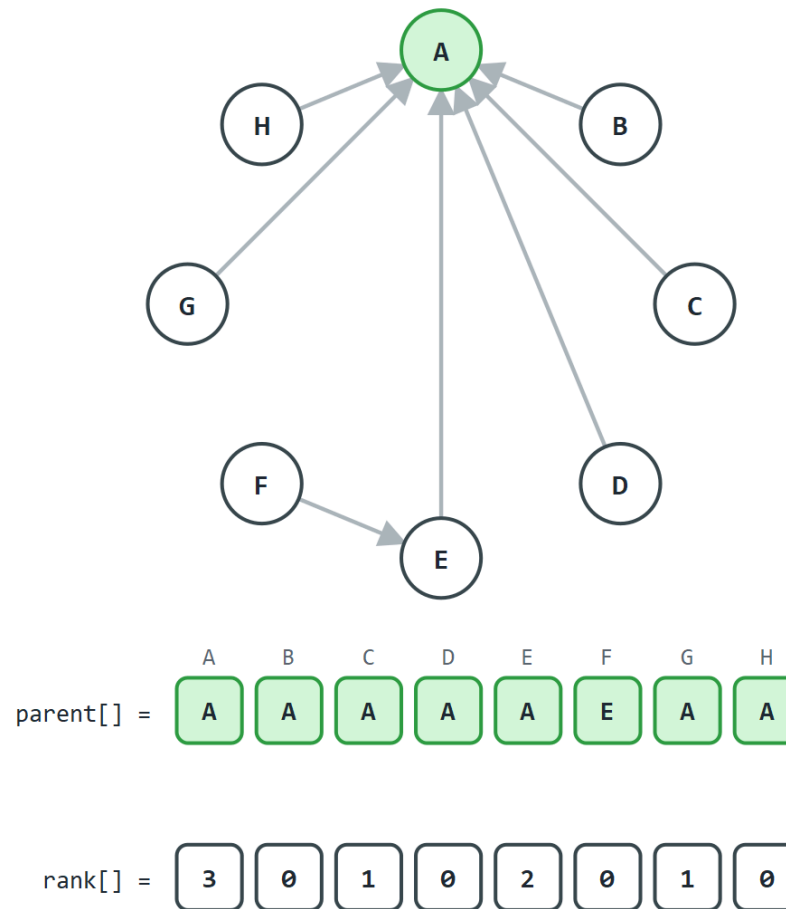
Union-find code (Java) — unionSets

```
static void unionSets(int a, int b) {  
    int ra=find(a), rb=find(b);  
    if (ra==rb) return;  
    if (rankOf[ra]<rankOf[rb]) parentOf[ra]=rb;  
    else if (rankOf[ra]>rankOf[rb]) parentOf[rb]=ra;  
    else { parentOf[rb]=ra; rankOf[ra]++; }  
}
```

Real output: union-find

```
union(A, B): merged, new root = A
union(C, D): merged, new root = C
union(A, C): merged, new root = A
find(D) = A
union(B, H): already the same set (A)
final sets: A->A B->A C->A D->A E->A F->A G->A H->A
```

Union-find: rank + path compression



Done. Sets: {A,B,C,D,E,F,G,H}.

Complexity & mistake

- $\sim O(1)$ amortised per operation, with both tricks
- Mistake: `union(a, b)` comparing `a`, `b` directly
- Must merge **roots** — `find(a)`, `find(b)` — never raw arguments

Mini question

Rank counts **height**, not the number of elements in a set. Why not just track set size instead?

(answer on the next slide)

Answer

Size works too, and is a common alternative — "union by size" hangs the smaller **set** under the larger one.

Both give the same $O(\alpha(n))$ guarantee; rank is the classical presentation because it directly bounds tree height, which is what **find** actually pays for.

4. Minimum Spanning Trees

A question to start

Connect n towns with power lines. Any pair could link directly, at a cost.

What is the cheapest set of lines that still connects everything?

A short history

- **Joseph Kruskal**, 1956
- **Robert Prim**, 1957 (rediscovering Jarník, 1930)
- Same problem, two structurally different greedy solutions
- Both provably safe: the **cut property**

Kruskal's idea

- Sort **all** edges by weight, once
- Scan cheapest-first; add an edge unless it **closes a cycle**
- Cycle check: union-find, near $O(1)$
- Disconnected graph → a spanning **forest**

Prim's idea

- Grow **one** tree from a start vertex
- Add the cheapest edge from **inside** to **outside**
- `key[v]` = cheapest edge connecting v to the tree so far
- Never reaches a different component: key stays "infinite"

Kruskal's code (C) — part 1/2

```
int kruskal_mst(Edge *sorted, int edge_count,
               Edge *mst_out, int *total_out) {
    for (int v=0; v<vertex_count; v++)
        { parent_of[v]=v; rank_of[v]=0; }
    qsort(sorted, edge_count, sizeof(Edge), cmp_weight);
    int mst_len=0, total=0;
```

Kruskal's code (C) — part 2/2

```
for (int i=0;i<edge_count;i++) {  
    if (find(sorted[i].a)==find(sorted[i].b))  
        continue;  
    union_sets(sorted[i].a, sorted[i].b);  
    mst_out[mst_len++]=sorted[i];  
    total+=sorted[i].w;  
}  
*total_out=total;  
return mst_len;  
}
```

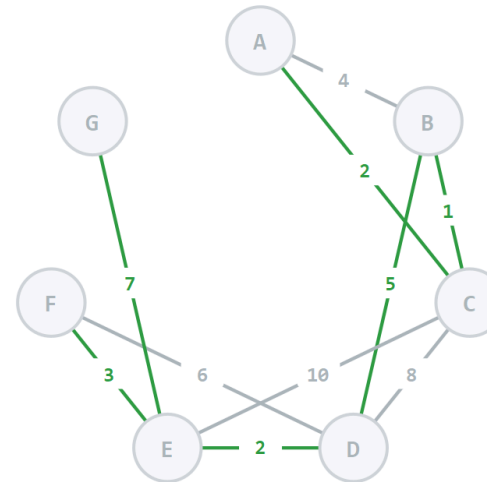
Kruskal's code (Java) — part 1/2

```
static int kruskalMst(Edge[] sorted,
                      Edge[] mstOut, int[] totalOut) {
    for (int v=0;v<vertexCount;v++)
        { parentOf[v]=v; rankOf[v]=0; }
    Arrays.sort(sorted, Comparator.comparingInt(
        (Edge e) -> e.w).thenComparingInt(e -> e.idx));
    int mstLen=0, total=0;
```

Kruskal's code (Java) — part 2/2

```
for (Edge e : sorted) {  
    if (find(e.a)==find(e.b)) continue;  
    unionSets(e.a, e.b);  
    mstOut[mstLen]=e; mstLen++;  
    total+=e.w;  
}  
totalOut[0]=total;  
return mstLen;  
}
```

Kruskal's MST



sorted edges =

B-C:1	A-C:2	D-E:2	E-F:3	A-B:4
B-D:5	D-F:6	E-G:7	C-D:8	C-E:10

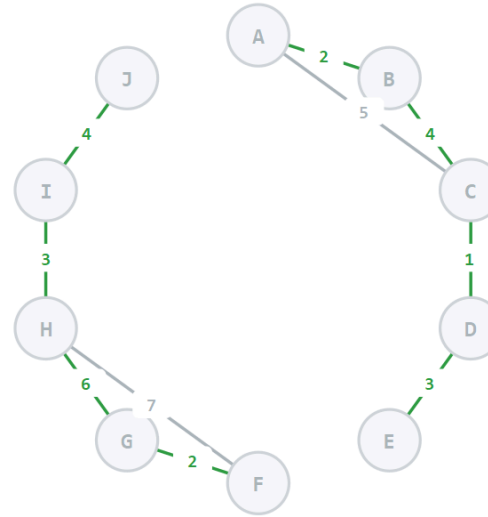
parent[] =

A	B	C	D	E	F	G
B	B	B	B	B	B	B

MST weight = 20

Done: 6 edges chosen, total weight 20. A single spanning tree.

Edge case: a spanning forest



sorted edges =

C-D:1	A-B:2	F-G:2	D-E:3	H-I:3	B-C:4
I-J:4	A-C:5	G-H:6	F-H:7		

parent[] =

A	B	C	D	E	F	G	H	I	J
A	A	A	C	C	F	F	F	H	H

MST weight = 25

Done: 8 edges chosen, total weight 25. The graph was not connected: the result is a spanning FOREST with 2 components.

Real output: Kruskal's MST

```
-- normal: 7 vertices, 10 edges, one component --  
MST edges: B-C:1 A-C:2 D-E:2 E-F:3 B-D:5 E-G:7  
total weight = 20  
components = 1  
  
-- edge: 10 edges, 2 components -- a spanning FOREST --  
MST edges: C-D:1 A-B:2 F-G:2 D-E:3 H-I:3 B-C:4 I-J:4 G-H:6  
total weight = 25  
components = 2 (a spanning forest)
```

Prim's code (C) — part 1/2

```
int prim_mst(Graph *g, int start,
             Edge *mst_out, int *total_out) {
    for (int v=0; v<g->vertex_count; v++)
        { key_of[v]=INF; in_mst[v]=0; parent_of[v]=-1; }
    key_of[start]=0;
    int mst_len=0, total=0;
    for (int count=0; count<g->vertex_count; count++) {
        int u=min_key_vertex(g->vertex_count);
        if (u==-1 || key_of[u]==INF) break;
        in_mst[u]=1;
```

Prim's code (C) — part 2/2

```
    if (parent_of[u] != -1) {
        mst_out[mst_len].a = parent_of[u];
        mst_out[mst_len].b = u;
        mst_out[mst_len].w = key_of[u];
        mst_len++; total += key_of[u];
    }
    for (AdjNode *n = g->adj[u]; n; n = n->next)
        if (!in_mst[n->to] && n->weight < key_of[n->to])
            { key_of[n->to] = n->weight;
              parent_of[n->to] = u; }
    }
    *total_out = total;
    return mst_len;
}
```

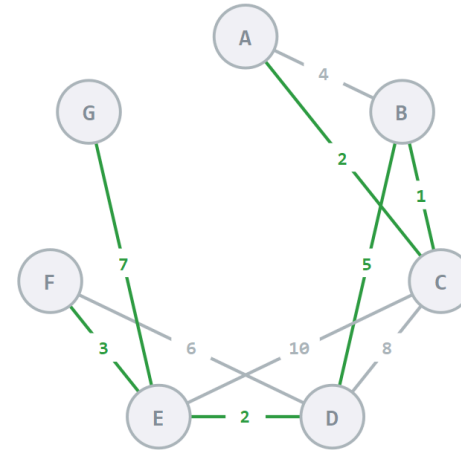
Prim's code (Java) — part 1/2

```
static int primMst(Graph g, int start,
                    Edge[] mstOut, int[] totalOut) {
    for (int v=0;v<g.vertexCount;v++)
        { keyOf[v]=INF; inMst[v]=false; parentOf[v]=-1; }
    keyOf[start]=0;
    int mstLen=0, total=0;
    for (int count=0;count<g.vertexCount;count++) {
        int u=minKeyVertex(g.vertexCount);
        if (u==-1 || keyOf[u]==INF) break;
        inMst[u]=true;
```

Prim's code (Java) — part 2/2

```
    if (parentOf[u] != -1) {
        mstOut[mstLen] = new Edge();
        mstOut[mstLen].a = parentOf[u];
        mstOut[mstLen].b = u;
        mstOut[mstLen].w = keyOf[u];
        mstLen++; total += keyOf[u];
    }
    for (AdjNode n = g.adj[u]; n != null; n = n.next)
        if (!inMst[n.to] && n.weight < keyOf[n.to])
            { keyOf[n.to] = n.weight; parentOf[n.to] = u; }
    }
    totalOut[0] = total;
    return mstLen;
}
```

Prim's MST



key[] =

A	B	C	D	E	F	G
0	1	2	5	2	3	7

parent[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

priority queue =

--	--	--	--	--	--	--

MST weight = 20

Done: 6 edges, total weight 20.

Real output: Prim's MST

```
-- normal: 7 vertices, 10 edges, starts at A --  
MST edges: A-C:2 C-B:1 B-D:5 D-E:2 E-F:3 E-G:7  
total weight = 20  
  
-- edge: 2 components, starts at A -- F..J never reached --  
MST edges: A-B:2 B-C:4 C-D:1 D-E:3  
total weight = 10  
unreached: F G H I J
```

Complexity table

Algorithm	Time	Note
Kruskal	$O(E \log E)$	sort dominates
Prim (array)	$O(V^2)$	matches Dijkstra's shape
Prim (heap)	$O(E \log V)$	better on dense graphs

Mini question

Could Prim ever pick a **more expensive** edge than Kruskal, on the same graph?

(answer: no — see next slide)

Answer

No. Both are provably optimal (cut property).

Every MST of a graph has the same **total** weight — they can differ only in *which* edges they pick, when weights tie.

5. Single-Source Shortest Paths

A question to start

A road network, weighted by distance. Starting from one city,
what is the cheapest way to reach every other city?

A short history

- **Edsger Dijkstra**, 1956 (published 1959)
- A 20-minute exercise to demo a new computer
- Still the standard answer when every weight is **non-negative**

Dijkstra's idea

- `dist[v]` = cheapest **total path** found so far
- Pick the not-yet-finished vertex with smallest `dist`
- Once picked: **final**, can never shrink again
- Only true because no edge weight is negative

Why negative weights break it

A vertex popped "final" early could later be beaten by a path through a very negative edge — discovered **after** it was already finalised.

Dijkstra's code (C) — part 1/2

```
void dijkstra(Graph *g, int start) {
    for (int v=0;v<g->vertex_count;v++)
        { dist_of[v]=INF; done[v]=0; parent_of[v]=-1; }
    dist_of[start]=0;
    for (int count=0;count<g->vertex_count;count++) {
        int u=min_dist_vertex(g->vertex_count);
        if (u==-1 || dist_of[u]==INF) break;
        done[u]=1;
    }
}
```

Dijkstra's code (C) — part 2/2

```
    for (AdjNode *n=g->adj[u]; n; n=n->next) {  
        int cand=dist_of[u]+n->weight;  
        if (!done[n->to] && cand<dist_of[n->to])  
            { dist_of[n->to]=cand; parent_of[n->to]=u; }  
    }  
}
```

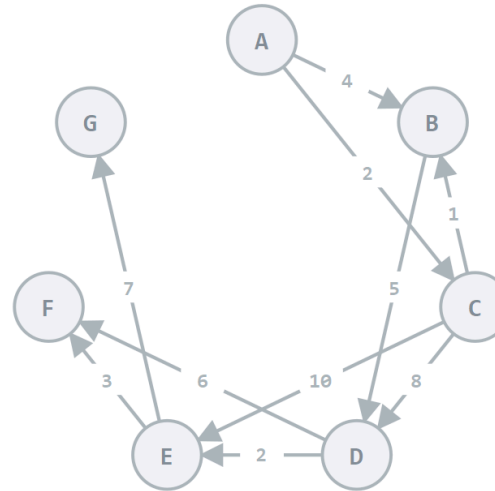
Dijkstra's code (Java) — part 1/2

```
static void dijkstra(Graph g, int start) {  
    for (int v=0;v<g.vertexCount;v++)  
        { distOf[v]=INF; done[v]=false; parentOf[v]=-1; }  
    distOf[start]=0;  
    for (int count=0;count<g.vertexCount;count++) {  
        int u=minDistVertex(g.vertexCount);  
        if (u==-1 || distOf[u]==INF) break;  
        done[u]=true;  
    }  
}
```

Dijkstra's code (Java) — part 2/2

```
    for (AdjNode n=g.adj[u]; n!=null; n=n.next) {  
        int cand=distOf[u]+n.weight;  
        if (!done[n.to] && cand<distOf[n.to])  
            { distOf[n.to]=cand; parentOf[n.to]=u; }  
    }  
}
```

Dijkstra's shortest path



dist[] =

A	B	C	D	E	F	G
0	3	2	8	10	13	17

parent[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

priority queue =

--	--	--	--	--	--	--

Done. Distances: A=0, B=3, C=2, D=8, E=10, F=13, G=17.

Real output: Dijkstra

```
-- normal: 8 vertices, 10 edges, starts at A --  
distances: A=0 B=3 C=2 D=8 E=10 F=13 G=17  
  
-- edge: F..J never reachable via the directed edges --  
distances: A=0 B=2 C=5 D=6 E=9 F=inf G=inf H=inf I=inf J=inf
```

Bellman-Ford: negative weights OK

- Bellman & Ford, late 1950s
- Give up "pop the minimum" entirely
- Relax **every** edge, fixed order, up to $V - 1$ rounds
- A V -th round that still improves = a **negative cycle**

Bellman-Ford's code (C) — part 1/2

```
int bellman_ford(Graph *g, int start) {
    for (int v=0;v<g->vertex_count;v++)
        { dist_of[v]=INF; parent_of[v]=-1; }
    dist_of[start]=0;
    for (int p=1;p<=g->vertex_count-1;p++) {
        int changed=0;
        for (int u=0;u<g->vertex_count;u++) {
            if (dist_of[u]==INF) continue;
```

Bellman-Ford's code (C) — part 2/2

```
        for (AdjNode *n=g->adj[u]; n; n=n->next) {
            int cand=dist_of[u]+n->weight;
            if (cand<dist_of[n->to])
                { dist_of[n->to]=cand;
                  parent_of[n->to]=u; changed=1; }
        }
    }
    if (!changed) break;
}
/* one more pass detects a negative cycle */
}
```

Bellman-Ford's code (Java) — part 1/2

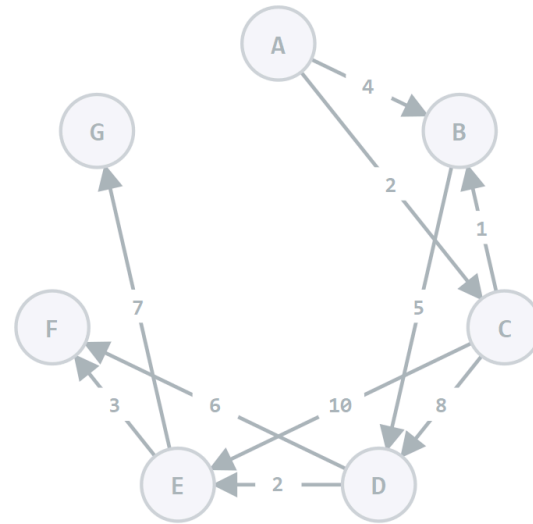
```
static boolean bellmanFord(Graph g, int start) {  
    for (int v=0;v<g.vertexCount;v++)  
        { distOf[v]=INF; parentOf[v]=-1; }  
    distOf[start]=0;  
    for (int pass=1;pass<=g.vertexCount-1;pass++) {  
        boolean changed=false;  
        for (int u=0;u<g.vertexCount;u++) {  
            if (distOf[u]==INF) continue;  

```

Bellman-Ford's code (Java) — part 2/2

```
        for (AdjNode n=g.adj[u]; n!=null; n=n.next) {
            int cand=distOf[u]+n.weight;
            if (cand<distOf[n.to])
                { distOf[n.to]=cand;
                  parentOf[n.to]=u; changed=true; }
        }
    }
    if (!changed) break;
}
/* one more pass detects a negative cycle */
}
```

Bellman-Ford shortest path



dist[] =

A	B	C	D	E	F	G
0	3	2	8	10	13	17

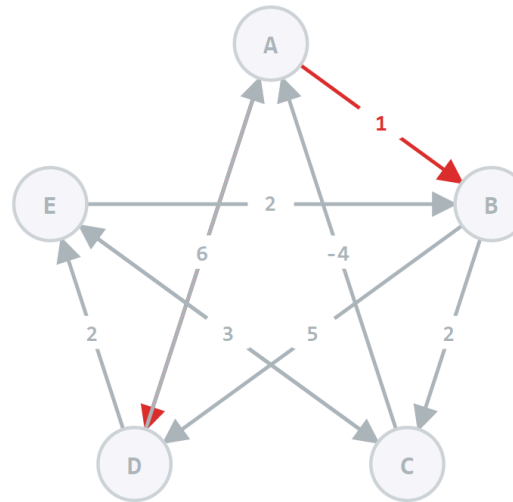
parent[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

round 3 / 6

Done. Distances: A=0, B=3, C=2, D=8, E=10, F=13, G=17. No negative cycle.

Required edge case: negative cycle



dist[] =

A	B	C	D	E
-4	-2	0	0	1

parent[] =

C	A	B	A	C
---	---	---	---	---

round 4 / 4

Done. Distances: A=-4, B=-2, C=0, D=0, E=1. There is a negative cycle.

Real output: Bellman-Ford

```
-- normal: 7 vertices, all positive, starts at A --  
distances: A=0 B=3 C=2 D=8 E=10 F=13 G=17  
no negative cycle  
  
-- edge: A-B-C-A is a negative cycle (total -1) --  
distances: A=-4 B=-2 C=0 D=0 E=1  
negative cycle detected
```

Complexity table

Algorithm	Time	Handles negatives?
Dijkstra	$O(V^2)$	No
Bellman-Ford	$O(V \cdot E)$	Yes

Mini question

Why exactly $V - 1$ rounds for Bellman-Ford, not V or $V/2$?

(answer on the next slide)

Answer

A shortest path (no negative cycle) never repeats a vertex — at most $V - 1$ edges.

Each round extends every path's "confirmed" prefix by one edge. $V - 1$ rounds confirm the longest possible shortest path.

6. All-Pairs Shortest Paths

A question to start

A flight-booking system needs the cheapest fare between **every pair** of ~20 cities, at once.

Running Dijkstra 20 times works — is there a way to **share** the work?

A short history

- **Robert Floyd & Stephen Warshall**, both 1962
- Independent, closely related matrix algorithms
- Combined algorithm carries both names

The idea

- An $N \times N$ matrix `dist[i][j]`
- For **every** vertex `k`: is `i -> k -> j` shorter than `dist[i][j]`?
- Try every vertex as an intermediate stop
- Update **in place** — safe, since `dist[i][k]` / `dist[k][j]` never change mid-pass

Floyd-Warshall's code (C)

```
void floyd_warshall(int vertex_cnt) {  
    for (int k=0;k<vertex_cnt;k++) {  
        for (int i=0;i<vertex_cnt;i++) {  
            for (int j=0;j<vertex_cnt;j++) {  
                if (dist[i][k]==INF || dist[k][j]==INF)  
                    continue;  
                int through=dist[i][k]+dist[k][j];  
                if (through<dist[i][j])  
                    dist[i][j]=through;  
            }  
        }  
    }  
}
```

Floyd-Warshall's code (Java)

```
static void floydWarshall(int vertexCnt) {  
    for (int k=0;k<vertexCnt;k++) {  
        for (int i=0;i<vertexCnt;i++) {  
            for (int j=0;j<vertexCnt;j++) {  
                if (dist[i][k]==INF || dist[k][j]==INF)  
                    continue;  
                int through=dist[i][k]+dist[k][j];  
                if (through<dist[i][j])  
                    dist[i][j]=through;  
            }  
        }  
    }  
}
```

Floyd-Warshall: the matrix fills in

	A	B	C	D	E
A	0	1	-3	2	-4
B	3	0	-4	1	-2
C	7	4	0	5	2
D	2	-1	-5	0	-3
E	8	5	1	6	0

Done: the shortest path for every pair of 5 vertices has been computed. No negative cycle.

Real output: Floyd-Warshall

```
-- normal: 5 vertices, negative edges but no negative cycle --
```

	A	B	C	E	D
A	0	1	-3	-4	2
B	3	0	-4	-2	1
C	7	4	0	2	5

```
no negative cycle
```

```
-- edge: A-B-C-A is a negative cycle --
```

```
negative cycle at: A B C
```

Complexity & mistake

- $O(V^3)$ time, $O(V^2)$ space
- Practical for a few hundred vertices, not more
- Mistake: `k` as the **innermost** loop silently computes garbage
- Mistake: forgetting the `== INF` guard → integer overflow

Mini question

Bellman-Ford finds negative cycles reachable **from one start**. Floyd-Warshall's diagonal check finds them differently — how?

(answer on the next slide)

Answer

Any `dist[v][v]` that drops **below zero** means: a path leaves `v` and comes back cheaper than staying put.

Since Floyd-Warshall computes **every** pair, this check runs for every vertex at once — no separate start vertex needed.

7. Strongly Connected Components

A question to start

Which groups of vertices can reach each other **and get back**, respecting edge direction?

Web pages linking in a cycle. Mutually-recursive functions.

Kosaraju's idea

- **Sergei Kosaraju**, ~1978 (credited via Micali & Vazirani, 1981)
- Phase 1: DFS, record every vertex's **finish time**
- Phase 2: DFS the **transpose** (edges reversed)
- Visit roots in **decreasing** finish-time order
- Each DFS tree in phase 2 = one SCC

Kosaraju's code (C)

```
void dfs1(int u) {           /* phase 1 */
    visited[u]=1;
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next)
        if (!visited[n->to]) dfs1(n->to);
    finish[finish_len++]=u;
}

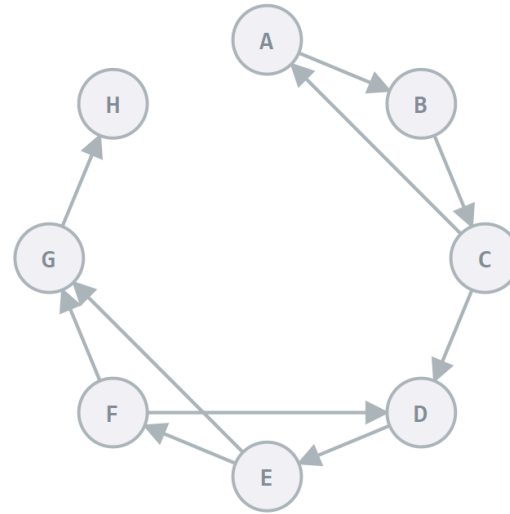
void dfs2(int u, int id) {    /* phase 2, on adjT */
    visited[u]=1; comp_of[u]=id;
    for (AdjNode *n=cur_g->adjT[u]; n; n=n->next)
        if (!visited[n->to]) dfs2(n->to, id);
}
```

Kosaraju's code (Java)

```
static void dfs1(int u) {           // phase 1
    visited[u]=1;
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next)
        if (visited[n.to]==0) dfs1(n.to);
    finish[finishLen]=u; finishLen++;
}

static void dfs2(int u, int id) {    // phase 2, on adjT
    visited[u]=1; compOf[u]=id;
    for (AdjNode n=curG.adjT[u]; n!=null; n=n.next)
        if (visited[n.to]==0) dfs2(n.to, id);
}
```

Strongly connected components



call stack =

--	--	--	--	--	--	--	--

finish order =

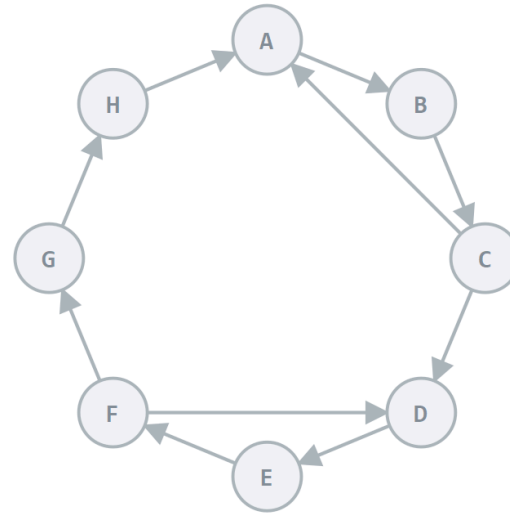
H	G	F	E	D	C	B	A
---	---	---	---	---	---	---	---

comp[] =

A	B	C	D	E	F	G	H
0	0	0	1	1	1	2	3

Done: 4 strongly connected components found: {A,B,C} {D,E,F} {G} {H}

Edge case: one big cycle



call stack =

--	--	--	--	--	--	--	--

finish order =

H	G	F	E	D	C	B	A
---	---	---	---	---	---	---	---

comp[] =

A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	0

Done: 1 strongly connected component found: {A,B,C,D,E,F,G,H}

Real output: strongly connected components

```
-- normal: 8 vertices, 2 cyclic components + 2 singletons --  
4 components:  
  A B C  
  D E F  
  G  
  H  
  
-- edge: everything is one big cycle --  
1 component:  
  A B C D E F G H
```

Complexity & mistake

- $O(V + E)$ — two full DFS passes
- Mistake: running phase 2 in the **same** order as phase 1
- Must be **reversed** — the single most common bug here

Mini question

On a **DAG** (no cycles at all), how many strongly connected components does Kosaraju's algorithm find?

(answer on the next slide)

Answer

Exactly **V** — one per vertex.

With no cycle anywhere, no vertex can return to itself through any path, so every component is a singleton. This is the last edge case in this week's SCC animation.

8. Bipartite Graphs

A question to start

Schedule exams so no student has two at once. Courses sharing a student $\rightarrow$ an edge.

Can this graph be 2-coloured — the simplest possible timetable?

BFS with two colours

- Colour the start `0` ; every neighbour the **opposite** colour
- A same-coloured neighbour already queued? **Conflict** — not bipartite
- One BFS per component (like Week 5's components loop)

The odd-cycle equivalence

A graph is bipartite if and only if it has no odd-length cycle.

An even cycle alternates colours perfectly. An odd one cannot close up consistently.

Bipartite check code (C) — part 1/2

```
int is_bipartite(Graph *g) {  
    for (int i=0; i<g->vertex_count; i++) color_of[i]=-1;  
    for (int s=0; s<g->vertex_count; s++) {  
        if (color_of[s]!=-1) continue;  
        color_of[s]=0; front=rear=0; enqueue(s);  
        while (front<rear) {  
            int u=dequeue();
```

Bipartite check code (C) — part 2/2

```
        for (AdjNode *n=g->adj[u]; n; n=n->next) {
            if (color_of[n->to]==-1)
                { color_of[n->to]=1-color_of[u];
                  enqueue(n->to); }
            else if (color_of[n->to]==color_of[u])
                return 0;
        }
    }
    return 1;
}
```

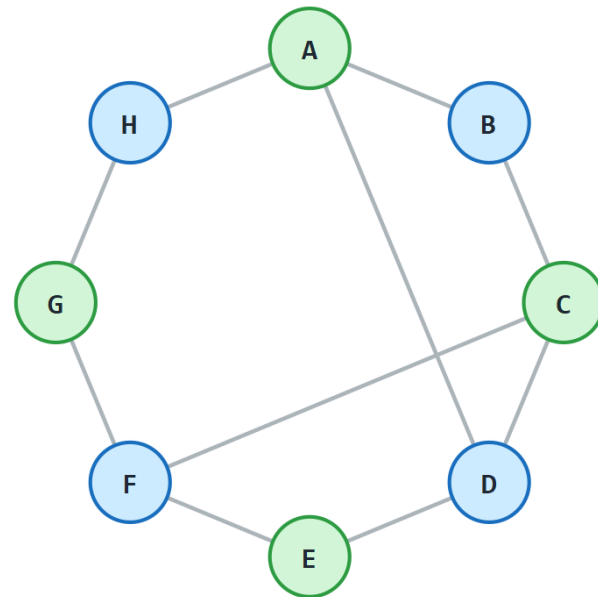
Bipartite check code (Java) — part 1/2

```
static boolean isBipartite(Graph g) {  
    for (int i=0;i<g.vertexCount;i++) colorOf[i]=-1;  
    for (int s=0;s<g.vertexCount;s++) {  
        if (colorOf[s]!=-1) continue;  
        colorOf[s]=0; front=rear=0; enqueue(s);  
        while (front<rear) {  
            int u=dequeue();
```

Bipartite check code (Java) — part 2/2

```
        for (AdjNode n=g.adj[u]; n!=null; n=n.next) {
            if (colorOf[n.to]==-1)
                { colorOf[n.to]=1-colorOf[u];
                  enqueue(n.to); }
            else if (colorOf[n.to]==colorOf[u])
                return false;
        }
    }
    return true;
}
```

Bipartite graph check



color[] =

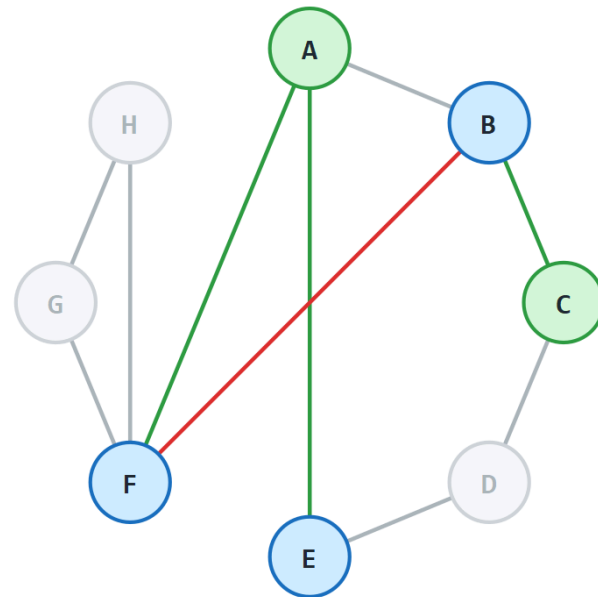
A	B	C	D	E	F	G	H
0	1	0	1	0	1	0	1

queue =

--	--	--	--	--	--	--	--

Done. Bipartite: the colour-0 side and the colour-1 side are never adjacent to each other.

Edge case: an odd cycle



color[] =

A	B	C	D	E	F	G	H
0	1	0	-	1	1	-	-

queue =

E	F	C					
---	---	---	--	--	--	--	--

Done. Not bipartite: an odd cycle exists.

Real output: bipartite check

```
-- normal: an even cycle plus 2 safe diagonals --  
colors: A=0 B=1 C=0 D=1 E=0 F=1 G=0 H=1  
bipartite  
  
-- edge: A-B-C-D-E-A is a 5-cycle (odd) --  
colors: A=0 B=1 C=0 D=-1 E=1 F=1 G=-1 H=-1  
NOT bipartite
```

Complexity & mistake

- $O(V + E)$ — one BFS
- Mistake: not handling **disconnected** graphs
- An odd cycle isolated in a second component is invisible without a per-component loop

Mini question

Why does a **conflict edge** ("same colour as its neighbour") always exist within one BFS layer or between two adjacent layers — never skip a layer?

(answer on the next slide)

Answer

BFS colours strictly by **distance parity** from the source — even distance gets colour 0, odd gets colour 1.

Any edge connects vertices whose BFS distances differ by exactly 0 or 1 (never 2+, or it wouldn't be a direct edge) — so a conflict can only appear exactly there.

9. Maximum Flow

A question to start

A water network: source, destination, pipes with capacities.

What is the largest total flow the network can deliver at once?

A short history

- **Ford & Fulkerson**, 1956 — the general method
- **Edmonds & Karp**, 1972 — always use the **shortest** augmenting path
- Proved: this choice guarantees polynomial time

The idea

- Find any **augmenting path**, s to t , with spare capacity
- Push the **bottleneck** (smallest capacity on the path)
- Pushing forward opens a **reverse** edge — a later path can "undo"
- The graph of "still usable" is the **residual graph**

Max-flow min-cut

Max flow **equals** the cheapest cut — the smallest total capacity separating s from t .

No augmenting path left = BFS's reachable set from s **is** that cut.

Edmonds-Karp code (C) — part 1/2

```
int edmonds_karp(int vertex_cnt, int s, int t) {  
    int max_flow=0;  
    while (bfs_augmenting_path(vertex_cnt, s, t)) {  
        int bottleneck=INF;  
        for (int v=t; v!=s; v=parent_of[v])  
            if (cap_of[parent_of[v]][v]<bottleneck)  
                bottleneck=cap_of[parent_of[v]][v];  
        max_flow+=bottleneck;  
    }  
    return max_flow;  
}
```

Edmonds-Karp code (C) — part 2/2

```
    for (int v=t; v!=s; v=parent_of[v]) {  
        int u=parent_of[v];  
        cap_of[u][v]-=bottleneck;  
        cap_of[v][u]+=bottleneck;  
    }  
    max_flow+=bottleneck;  
}  
return max_flow;  
}
```

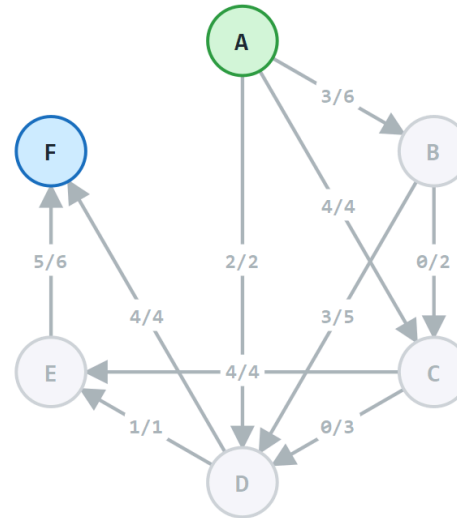
Edmonds-Karp code (Java) — part 1/2

```
static int edmondsKarp(int vertexCnt, int s, int t) {  
    int maxFlow=0;  
    while (bfsAugmentingPath(vertexCnt, s, t)) {  
        int bottleneck=INF;  
        for (int v=t; v!=s; v=parentOf[v]) {  
            int u=parentOf[v];  
            if (capOf[u][v]<bottleneck)  
                bottleneck=capOf[u][v];  
        }  
    }
```

Edmonds-Karp code (Java) — part 2/2

```
        for (int v=t; v!=s; v=parentOf[v]) {  
            int u=parentOf[v];  
            capOf[u][v]-=bottleneck;  
            capOf[v][u]+=bottleneck;  
        }  
        maxFlow+=bottleneck;  
    }  
    return maxFlow;  
}
```

Edmonds-Karp maximum flow



residual =

A>B:3	B>A:3	B>C:2	B>D:2	C>A:4
C>D:3	D>A:2	D>B:3	E>C:4	E>D:1
E>F:1	F>D:4	F>E:5		

max flow = 9

Done. Maximum flow from A to F = 9 (4 augmenting paths used).

Real output: Edmonds-Karp

```
-- normal: 6 vertices, 10 edges, A to F --  
max flow from A to F = 9  
  
-- edge: A and J are in two separate components --  
max flow from A to J = 0
```

Complexity & mistake

- $O(V * E^2)$ for Edmonds-Karp specifically
- Mistake: forgetting the **reverse** edge update
- Without it: plain Ford-Fulkerson, can get stuck short of optimal

Mini question

Section 3's union-find could answer "0 flow, s and t disconnected" **instantly**. Why does this section still run a full BFS first?

(answer on the next slide)

Answer

Union-find only knows **reachability**, not **capacity**.

Even when s and t ARE connected, the actual max flow depends on the bottleneck capacities along the way — something union-find's parent pointers never recorded.

10. Backtracking

A question to start

Colour a map so no two neighbours share a colour, with as few colours as possible.

No known formula. How do you search systematically?

The idea: try, recurse, undo

- Extend a partial solution by one decision
- Still consistent? **Recurse** and keep extending
- Inconsistent, or every extension fails? **Undo** and try the next option

Why graph colouring, not Hamiltonian path?

Colouring reuses Section 8's exact `color[]` row and neighbour-conflict check.

A Hamiltonian path needs an entirely new "path so far" convention — for comparatively little extra insight.

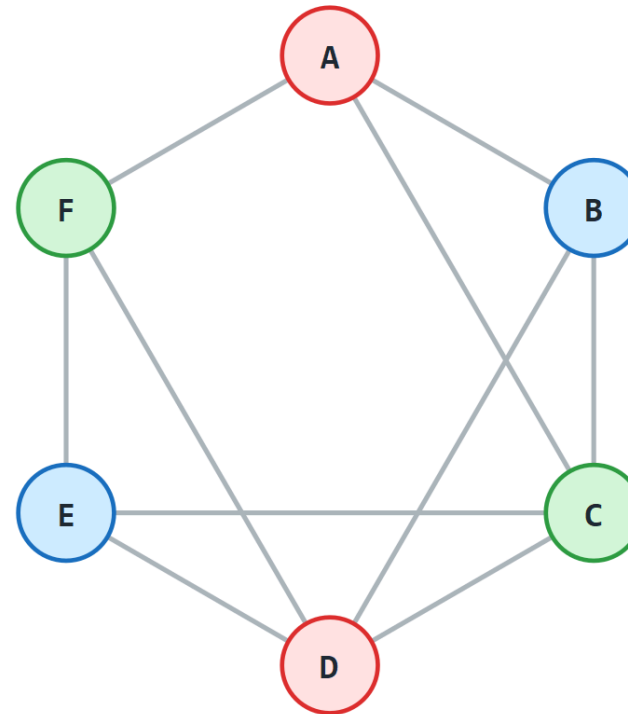
Backtracking code (C)

```
int color_graph(int v, int k) {  
    if (v==cur_g->vertex_count) return 1;  
    for (int c=1;c<=k;c++) {  
        if (safe(v,c)) {  
            color_of[v]=c;  
            if (color_graph(v+1,k)) return 1;  
            color_of[v]=0; /* backtrack */  
        }  
    }  
    return 0;  
}
```

Backtracking code (Java)

```
static boolean colorGraph(int v, int k) {  
    if (v==curG.vertexCount) return true;  
    for (int c=1;c<=k;c++) {  
        if (safe(v,c)) {  
            colorOf[v]=c;  
            if (colorGraph(v+1,k)) return true;  
            colorOf[v]=0;    // backtrack  
        }  
    }  
    return false;  
}
```

Backtracking: graph colouring

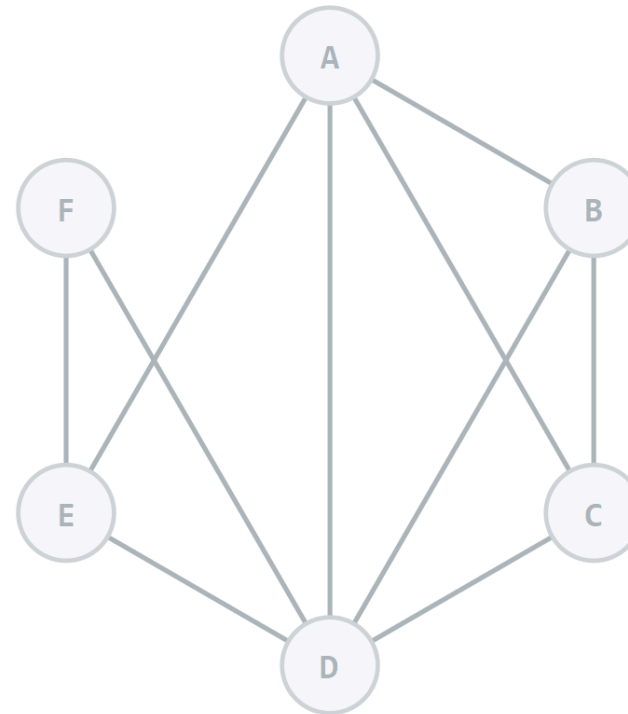


color[] =

A	B	C	D	E	F
1	2	3	1	2	3

Done. Colouring: A=1, B=2, C=3, D=1, E=2, F=3.

Required edge case: K4, unsolvable



color[] =

A	B	C	D	E	F
0	0	0	0	0	0

Done. 3 colours are not enough.

Real output: backtracking graph colouring

```
-- normal: 6 vertices, 10 edges, k=3 --  
colouring: A=1 B=2 C=3 D=1 E=2 F=3  
  
-- edge: K4 is UNSOLVABLE with k=3 --  
no valid colouring with k=3
```

Complexity & mistake

- Worst case $O(k^V)$ — a last-resort technique
- `safe()` prunes huge swaths early — why it's usable at all
- Mistake: forgetting the **undo** step corrupts later branches

Mini question

K4 needs 4 colours. How many **vertices** does the algorithm actually try to colour before reporting "unsolvable" with $k=3$?

(answer on the next slide)

Answer

All of them, **every time** it backtracks past vertex 0 — but the failure is detected at vertex 3, the 4th vertex of K4, since the first 3 already used all 3 available colours between them.

`safe()` then rejects every colour for vertex 3, forcing a chain of undos all the way back to the start.

How each program was checked

- Every C program compiles with `-Wall -Wextra -Werror`
- Every Java program compiles with `-Xlint:all -Werror`
- C and Java output differed **byte-identical**, same scenarios
- Every unit test file also rebuilt under **AddressSanitizer**

One sentence per algorithm

- **Kahn / DFS topo-sort**: order respecting every "before" rule
- **Cycle detection**: which vertices, not just whether
- **Union-find**: "same group?" in near-constant time
- **Kruskal / Prim**: cheapest way to connect everything
- **Dijkstra / Bellman-Ford**: cheapest path from one start
- **Floyd-Warshall**: cheapest path between every pair at once

One sentence per algorithm (cont.)

- **Kosaraju SCC**: mutual reachability, directed
- **Bipartite check**: can this be split into exactly two teams?
- **Edmonds-Karp**: the largest flow a network can carry
- **Backtracking**: try, recurse, undo — for problems with no formula

Summary table (1/3)

Problem	Algorithm	Time
Order	Kahn / DFS topo-sort	$O(V+E)$
Cycle?	3-colour DFS	$O(V+E)$
Groups	Union-find	$\sim O(1)$ amortised

Summary table (2/3)

Problem	Algorithm	Time
Cheapest connect	Kruskal / Prim	$O(E \log E) / O(V^2)$
Cheapest path	Dijkstra / Bellman-Ford	$O(V^2) / O(V \cdot E)$
All-pairs paths	Floyd-Warshall	$O(V^3)$

Summary table (3/3)

Problem	Algorithm	Time
Mutual reach	Kosaraju SCC	$O(V+E)$
2-team split	Bipartite check	$O(V+E)$
Max flow	Edmonds-Karp	$O(V \cdot E^2)$
No formula	Backtracking	$O(k^V)$ worst case

Exercises (pick a few)

- Trace Kahn's algorithm by hand on a 5-edge DAG
- Build a graph with a negative edge but no negative cycle
- Modify SCC to flag trivial (single-vertex) components
- Prove every tree is bipartite

Self-check: 3 quick questions

1. Why does Dijkstra fail on a negative edge?
2. What's the ONE new idea Kosaraju adds over topological-sort DFS?
3. What does a reverse residual edge let a later augmenting path do?

Looking ahead

Week 10: Advanced Tree Structures

AVL trees, red-black trees, B-trees — the self-balancing machinery that keeps $O(\log n)$ from collapsing to $O(n)$ on unlucky insertions.



Questions?

CEN207 Data Structures · Week 9 · Graph Algorithms